HTTP Client Experiment

1. HTTP Client Overview

HTTP (HyperText Transfer Protocol) is the most widely used application layer protocol on the Internet. It is based on TCP and uses a **request-response** model: the client sends a GET request, and the server returns a status code + data. In this experiment the ESP32-S3 acts as an HTTP client, constructing and sending a GET request to the public server `example.com` using a pure socket, verifying network reachability.

Typical application scenarios:

Application Type	Example
API calls	Report sensor data to a cloud platform REST API
Weather fetching	Request a weather API to get real-time weather
OTA firmware	Download firmware update files
Webhook	Trigger IFTTT / DingTalk robot notifications

2. Core Concepts

2.1 HTTP Request Format

```
GET / HTTP/1.1\r\n
Host: example.com\r\n
Connection: close\r\n
\r\n
```

Request line + headers + blank line, three parts separated by `\r\n`.

2.2 HTTP Response Format

```
HTTP/1.1 200 OK\r\n           <- status line
Content-Type: text/html\r\n      <- headers
\r\n
<html>...</html>                <- response body
```

2.3 Configuration Parameters

Parameter	Description	Value Used in This Experiment
HOST	Target server	`example.com`
PORT	HTTP port	`80`

---

3. Hardware Dependencies

The ESP32-S3 has built-in WiFi and needs access to the public Internet (example.com).

---

4. Project Architecture and Flow

```
Script execution (single-file .py)
  |
  |- wifi_connect(): connect to WiFi (STA mode)
  |
  |- GET / -> DNS resolution -> TCP connection -> send request -> receive
response -> print -> end
```

---

5. Source Code Explained

Full source code: `Source Code -> MicroPython -> 4.Network Protocols -> HTTP_Client -> HTTP_Client.py`

5.1 Main Program

`getaddrinfo` resolves the domain -> `connect` establishes a TCP connection -> build the HTTP request headers -> `sendall` sends -> loop `recv` to receive the response until the connection closes. After one execution it exits.

```python
def main():
    wifi_connect(WIFI_SSID, WIFI_PASS)   # connect to WiFi

    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # TCP socket
    sock.settimeout(10)                  # 10s timeout

    ip = socket.getaddrinfo(HOST, PORT)[0][4][0] # DNS resolve
    sock.connect((ip, PORT))             # connect
```

```
    req = f"GET / HTTP/1.1\r\nHost: {HOST}\r\nConnection: close\r\n\r\n"  #
request line + headers
    sock.sendall(req.encode("utf-8"))    # send

    resp = b""
    while True:
        chunk = sock.recv(512)           # receive
        if not chunk:                    # closed
            break
        resp += chunk
    sock.close()                         # close

    text = resp.decode("utf-8")
    _, _, body_text = text.partition("\r\n\r\n")# split body
    status = int(text.split(" ")[1]) if " " in text else 0  # status code

    print(f"[HTTP] <<< {status}  {len(resp)}B")
    print(body_text[:800])              # first 800 chars
```

6. Operating Procedures

6.1 Flashing and Running

Thonny IDE steps: See `MicroPython -> 1. Before Development -> 3.Thonny IDE`.

1. Modify `WIFI_SSID` / `WIFI_PASS`

```
WIFI_SSID    = "Yahboom2"              # WiFi名称 / WiFi SSID
WIFI_PASS    = "yahboom890729"         # WiFi密码 / WiFi password

HOST = "example.com"                   # 目标主机 / target host
PORT = 80                              # HTTP 端口
```

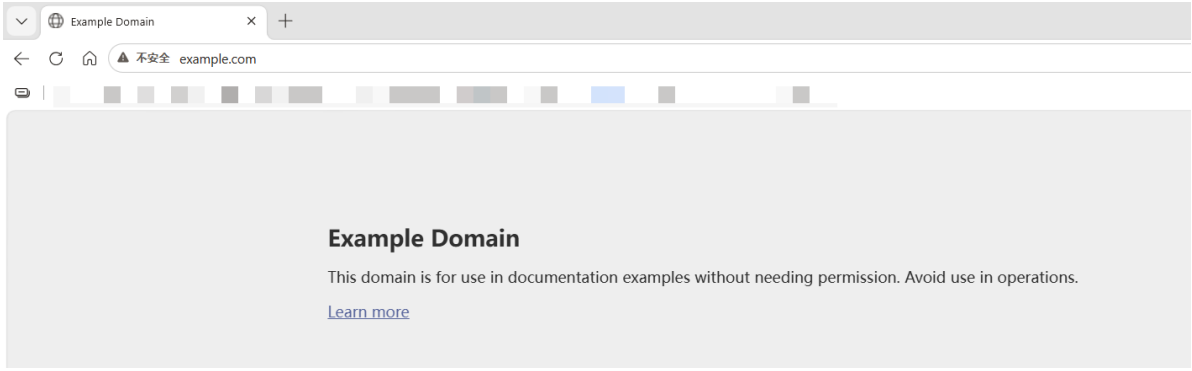
2. Run `HTTP_Client.py`; it exits after performing one GET

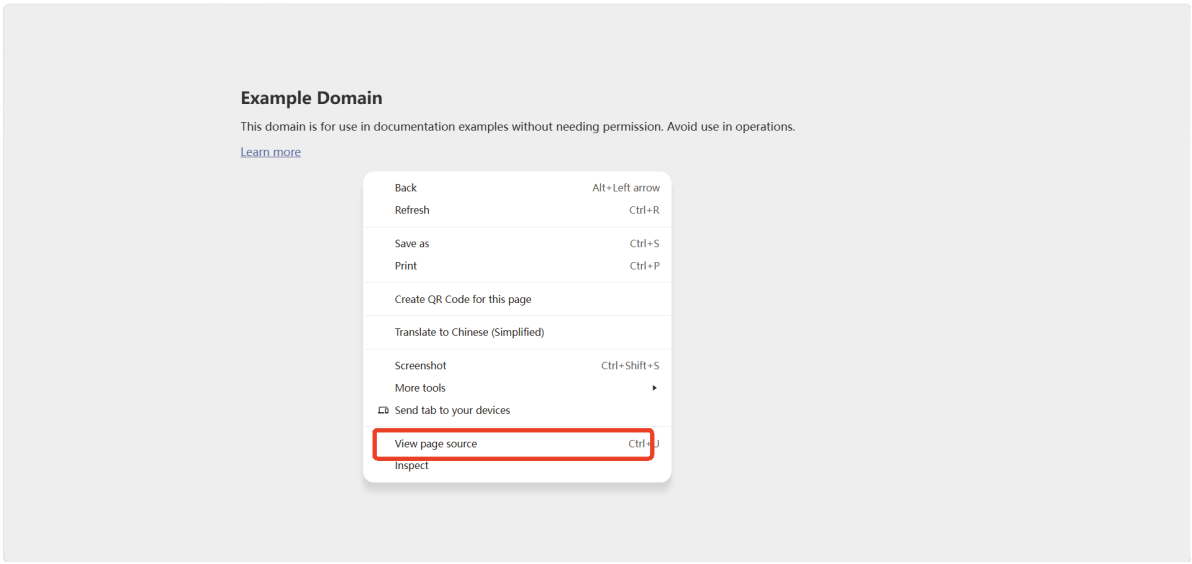
6.2 Serial Output Example

```
===== ESP32-S3 HTTP Client =====
[WiFi] Already connected. IP: 192.168.11.229
GET http://example.com/

[HTTP] >>> GET /
[HTTP] <<< 200  869B
22f
<!doctype html><html lang="en"><head><title>Example Domain</title><link
rel="icon" href="data:,"><meta name="viewport" content="width=device-width,
initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-
family:system-ui,sans-serif}h1{font-
size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div>
<h1>Example Domain</h1><p>This domain is for use in documentation examples
without needing permission. Avoid use in operations.</p><p><a
href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
```

```
===== ESP32-S3 HTTP Client =====
[WiFi] Already connected. IP: 192.168.11.229
GET http://example.com/

[HTTP] >>> GET /
[HTTP] <<< 200  869B
22f
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="vi
ewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;f
ont-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><
body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission
. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></ht
ml>
```

The HTML content printed in the middle of the serial output is the HTML returned by http://example.com. You can visit this URL in a browser, right-click -> **View Page Source** (or `Ctrl+U`), and compare it with the serial output: if they match, the GET request succeeded.





6.3 Verification Methods

Operation	Expected Result
Run the script	Prints one GET response with status code 200

7. Common Problems

Symptom	Cause	Solution
`getaddrinfo` fails	DNS cannot resolve	Check whether the WiFi connection was successful
Status code is not 200	Malformed request	Check the HTTP header format (`\r\n` separated)
Response body truncated	`recv()` loop exits early	Ensure `while True` until `chunk == b""`